

AI Software Supply-Chain Security and Vulnerability Detection: A Comprehensive Survey

Abstract

The integration of Artificial Intelligence (AI) into software development lifecycles has fundamentally altered the topology of the software supply chain, introducing novel attack surfaces while simultaneously offering powerful mechanisms for defense. This survey provides a comprehensive analysis of the state-of-the-art in AI software supply-chain security and vulnerability detection, synthesizing evidence from 300 academic papers published between 2014 and 2026. We categorize the field into three primary domains: the structural characterization of AI-specific supply chains, the detection of vulnerabilities and malicious artifacts using machine learning and agentic systems, and the governance frameworks required to secure these ecosystems. Our analysis reveals that while traditional static analysis tools struggle with the dynamic nature of AI dependencies and model serialization formats, emerging neuro-symbolic frameworks, multi-agent orchestration systems, and dynamic behavioral monitoring offer significant improvements in detection accuracy. However, critical challenges remain regarding dataset bias, the adversarial robustness of detection models, and the gap between SBOM generation and consumption. This paper systematically reviews methodological families ranging from graph-based dependency analysis to large language model (LLM)-driven code auditing, highlighting the evolution from reactive patching to proactive, autonomous defense mechanisms.

1. Introduction

The software supply chain has evolved from a linear sequence of code compilation and distribution into a complex, hyper-connected network of open-source libraries, pre-trained models, datasets, and automated build pipelines. The advent of “Software 2.0,” where neural networks and probabilistic models constitute core system components, has exacerbated existing security vulnerabilities and introduced unique risks such as model poisoning, serialization exploits, and dependency confusion in AI registries [1], [2]. Recent high-profile incidents, including the XZ Utils backdoor and widespread protests in open-source repositories, underscore the fragility of trust mechanisms in modern software ecosystems [3], [4].

The motivation for this survey stems from the dual role of AI in supply chain security: it acts as both a potent attack vector and a critical defensive tool. On one hand, AI systems introduce new layers of opacity; trained artifacts encode behavior via stochastic processes, breaking deterministic hash verification assumptions and complicating provenance tracking [1]. On the other hand, AI-driven techniques, particularly those leveraging Large Language Models (LLMs) and Graph Neural Networks (GNNs), have demonstrated superior capabilities in detecting subtle code anomalies, obfuscated malware, and cascading vulnerability chains that evade traditional rule-based scanners [5], [6].

The evolution of this field can be traced through three distinct phases. Initially, research focused on empirical measurements of dependency networks, revealing that a significant portion of open-source software relies on vulnerable transitive dependencies [7], [8]. Subsequently, the focus shifted to machine learning-based vulnerability detection, where deep learning models began outperforming static analyzers in identifying semantic flaws in source code and binaries [9], [10]. Most recently, the field has entered an era of agentic AI, where autonomous multi-agent systems orchestrate complex reasoning tasks to assess risk, generate patches, and enforce security policies across heterogeneous

environments [11], [12].

Despite these advancements, significant gaps persist. Industry summits consistently report a disconnect between the ease of generating Software Bills of Materials (SBOMs) and the difficulty of consuming them for actionable risk mitigation [13], [14]. Furthermore, the reliance on synthetic datasets for training vulnerability detectors has led to models that overfit to specific augmentation strategies rather than learning generalizable semantic features [15], [16]. This survey aims to synthesize the current landscape, providing a structured taxonomy of methodological approaches, evaluating their efficacy based on empirical evidence, and outlining the critical challenges that must be addressed to secure the future of AI-enabled software supply chains.

2. Method

The methodology for securing AI software supply chains encompasses a diverse array of techniques, ranging from static dependency graph analysis to dynamic runtime monitoring and autonomous agentic reasoning. We organize the existing literature into coherent methodological families: Supply Chain Topology and Risk Characterization, Static and Dynamic Vulnerability Detection, AI-Driven and Agentic Security Systems, Model and Binary Analysis, and Governance Frameworks including SBOMs and Provenance.

2.1. Supply Chain Topology and Risk Characterization

Understanding the structural properties of the software supply chain is a prerequisite for effective risk management. Recent empirical studies have moved beyond simple dependency counting to analyze the topology, propagation dynamics, and specific vulnerabilities inherent in AI-centric ecosystems.

2.1.1. Dependency Graph Analysis and Propagation Dynamics The structure of software dependencies in AI ecosystems exhibits unique characteristics compared to traditional software. Research indicates that the LLM supply chain is locally dense but globally sparse, dominated by a few large trees where vulnerabilities can cascade rapidly [17]. In the Python ecosystem, vulnerable dependencies often reside at deeper transitive levels, with an average depth of 4.1 for guaranteed exposures, making them difficult to detect via surface-level scanning [18]. Similarly, in the npm ecosystem, indirect dependencies constitute the primary exposure vector, with 36.5% of packages exposed indirectly compared to direct dependencies [8].

Graph-based modeling has emerged as a critical tool for capturing these structural connections. Heterogeneous Graph Attention Networks (HGAT) have been employed to model SBOM components and vulnerabilities as nodes, enabling the prediction of cascading vulnerability chains that isolated analysis misses [19], [20]. These models leverage relations such as `DEPENDS_ON` and `HAS_VULNERABILITY` to learn node representations, achieving accuracy rates exceeding 91% in node classification tasks [19]. Furthermore, time-dependent bipartite graphs have been used to model developer interactions, revealing that fragile team dynamics serve as behavioral enablers for social engineering attacks, which are a precursor to supply chain compromises [21].

2.1.2. AI-Specific Supply Chain Risks The integration of AI models introduces distinct risk vectors not present in traditional code. The “Grand Software Supply Chain of AI Systems” framework decomposes the supply chain into four layers: data acquisition, model training, model

inference, and cross-cutting substrate, identifying integrity gaps specific to each [1]. A major concern is the prevalence of unsafe serialization formats; studies show that 59% of analyzed model files on Hugging Face utilize unsafe formats like Pickle, with 96% of these being successfully exploitable via object injection [22]. Malicious actors exploit dataset loading scripts and hidden APIs within frameworks like TensorFlow to execute arbitrary code, bypassing standard scanners [23], [24].

Moreover, the dependency practices in LLM repositories reveal significantly longer undisclosed vulnerability periods (median 56.2 months) compared to the broader Python ecosystem, indicating a lag in security awareness and remediation [25]. The “bloat” in ML containers, where up to 80% of container size consists of unused libraries, further expands the attack surface by including unnecessary vulnerable components [26].

2.1.3. Weak Link Signals and Actor Reputation Identifying weak links in the supply chain is crucial for proactive defense. Frameworks have been developed to quantify “weak link signals” such as expired maintainer domains, which expose thousands of packages to hijack risks [27]. The concept of “Technical Leverage,” defined as the ratio of third-party dependency code size to own code, has been formalized to assess risk; small-to-medium libraries often exhibit disproportionately high leverage, correlating with increased vulnerability odds [28].

Reputation systems are also being integrated into risk assessment. The Actor Reputation Metric System (ARMS) operationalizes trust by quantifying contributor history and adherence to security standards, successfully identifying low-reputation actors in retrospective case studies like the XZ Utils backdoor [29]. Similarly, metadata-driven approaches analyze maintainer reputation and documentation completeness to filter false positives in typosquatting detection, reducing false positive rates by 52% compared to prior state-of-the-art methods [30].

2.2. Static and Dynamic Vulnerability Detection

Vulnerability detection remains the cornerstone of supply chain security. The field has transitioned from purely rule-based static analysis to hybrid approaches incorporating deep learning, dynamic tracing, and semantic reasoning.

2.2.1. Deep Learning for Source Code Analysis Deep learning models, particularly Graph Neural Networks (GNNs) and Transformers, have become dominant in source code vulnerability detection. Graph-based models, which represent code as Code Property Graphs (CPG) integrating Abstract Syntax Trees (AST), Control Flow Graphs (CFG), and Program Dependency Graphs (PDG), consistently outperform sequential models by capturing semantic dependencies [31], [32]. For instance, models utilizing CPGs with Convolutional Neural Networks (CNNs) have achieved accuracy rates of 92% in detecting vulnerabilities in C/C++ code [31].

However, the choice of representation and training strategy significantly impacts performance. Studies indicate that encoder-only architectures with extended context windows outperform decoder-only models by capturing bidirectional structural dependencies, achieving F1-scores of 94.73% on benchmark datasets [33]. Conversely, reliance on synthetic datasets has been identified as a major pitfall; models trained on synthetic data often fail to generalize to real-world code, with performance dropping sharply when evaluated on out-of-distribution datasets like recent Linux kernel CVEs [34], [35]. To address class imbalance, a pervasive issue in vulnerability datasets, techniques such as Focal Loss and adversarial augmentation have been explored, though no single solution dominates across all metrics [36].

2.2.2. Dynamic Analysis and Behavioral Profiling Dynamic analysis complements static methods by observing runtime behavior, which is particularly effective for detecting obfuscated malware and install-time attacks. Frameworks like eDySec and DySec utilize eBPF probes to capture system calls, network traffic, and file access patterns during package installation in the PyPI ecosystem. By extracting 36 real-time features, these systems achieve accuracy rates of 99% and 95.99% respectively, significantly reducing false negatives compared to static analysis [37], [38].

The QUT-DV25 dataset, constructed using isolated Raspberry Pi nodes and eBPF tracing, provides a robust benchmark for dynamic analysis, demonstrating that combining heterogeneous trace types yields superior detection accuracy [39]. Furthermore, function-level fuzz testing with static type inference has been employed to construct parameters for exported objects, achieving an F1-score of 0.95 in detecting malicious npm packages [40]. Despite these successes, dynamic analysis faces limitations regarding platform dependency (often restricted to Linux) and the inability to detect time-delayed payloads or attacks requiring specific user interactions [37], [40].

2.2.3. Hybrid and Neuro-Symbolic Approaches To overcome the limitations of purely statistical or rule-based methods, hybrid and neuro-symbolic frameworks are gaining traction. These approaches combine the precision of symbolic reasoning with the flexibility of neural networks. For example, the SCAA framework maps stripped binaries to abstract security lattices using Reflexive Prompting, enforcing Galois Connection soundness to bound LLM hallucinations, achieving an F1-score of 89.4% [41].

Similarly, taint-based code slicing has been adapted for LLM-based detection, reducing input volume by over 99% while retaining 73.7% of malicious features, resulting in an F1-score of 83.32% for npm package detection [42]. In the realm of smart contracts, frameworks like CrossInspector employ Transformer models to recover state variable semantics from decompiled bytecode, constructing State Dependency Graphs that achieve precision and recall rates above 96% [43]. These hybrid methods demonstrate that integrating structural verification with semantic analysis is key to handling the complexity of modern software artifacts.

2.3. AI-Driven and Agentic Security Systems

The most recent paradigm shift involves the use of AI not just as a classifier, but as an autonomous agent capable of reasoning, planning, and executing complex security tasks. This section covers tool learning, function calling, and multi-agent orchestration.

2.3.1. LLM-Based Code Auditing and Rule Generation Large Language Models (LLMs) have shown remarkable potential in auditing code and generating security rules. Systems like SocketAI utilize iterative self-refinement workflows where LLMs act as security researchers to validate findings, achieving a precision of 99% in detecting malicious npm packages, significantly outperforming traditional static analysis tools like CodeQL [44]. Similarly, RULELLM employs a three-stage pipeline (Crafting, Refining, Aligning) to automatically generate YARA and Semgrep rules from malware metadata, achieving an F1-score of 88.4% [45].

However, LLMs are not without limitations. Studies reveal that LLMs often fail at reasoning stages required for vulnerability detection, such as localizing relevant statements and understanding pointer operations, with balanced accuracy hovering near random guessing (54.5%) on complex reasoning tasks [46]. To mitigate this, Retrieval-Augmented Generation (RAG) has been explored, though counterintuitively, fine-tuning LLMs on behavioral feature descriptions has been found to

yield superior accuracy (97%) compared to RAG-augmented prompts for PyPI malware detection [47].

2.3.2. Multi-Agent Orchestration and Autonomous Defense Multi-agent systems (MAS) represent the frontier of autonomous defense, orchestrating specialized agents to handle different aspects of security assessment. The LibVulnWatch system orchestrates specialized agents via a Directed Acyclic Graph (DAG) to assess five risk domains in parallel, covering up to 88% of applicable OpenSSF Scorecard checks [5]. Similarly, the CHASE framework implements a Plan-and-Execute workflow with a Supervisor agent and specialized Worker agents (e.g., Deobfuscator, Web Researcher), achieving a recall of 98.4% in dissecting malicious PyPI packages [12].

Autonomous defense frameworks integrate LLM reasoning with Reinforcement Learning (RL) to optimize vulnerability mitigation. One such framework employs PPO-based RL agents alongside LLMs to perform semantic vulnerability analysis, achieving an autonomous mitigation rate of over 90% in real CI/CD pipelines [11]. These systems leverage chain-of-thought prompting and iterative evidence gathering to synthesize disparate information sources, identifying complex risks like Remote Code Execution (RCE) and regulatory gaps that static tools miss [5], [11].

2.3.3. Agentic Tool Use and Planning Effective agentic systems rely on robust tool use and planning capabilities. Frameworks like Argus implement LLM-centered workflows with Retrieval-Augmented Generation (RAG) to perform full-chain security analysis, significantly increasing vulnerability detection counts in industrial codebases compared to baselines [6]. The VulnAgent-X framework employs a layered agentic sequential chain, including a “Sceptic Agent” that explicitly searches for counter-evidence, framing detection as a staged evidence accumulation process to reduce false positives [48].

Planning for tool use is critical in these systems. The DepDec-Bench framework evaluates AI coding agents on dependency decision-making, revealing that agents select known-vulnerable versions 2.46% of the time, highlighting the need for improved planning algorithms that incorporate security policies [49]. Furthermore, agentic workflows are being extended to binary analysis, where agents summarize binary diffs using Functional Sensitivity Scores to triage sensitive functions, although recall remains a challenge for smaller models [50].

2.4. Model and Binary Analysis

Securing the supply chain requires rigorous analysis of both compiled binaries and AI models, which often lack source code or exist in opaque formats.

2.4.1. Binary Vulnerability Detection Binary analysis has advanced through the application of deep learning to machine code and intermediate representations (IR). Techniques that tokenize x86-64 machine code while omitting operand bytes to reduce vocabulary size have enabled 1D-CNN and GCN architectures to detect vulnerabilities with high accuracy [51]. Graph-based models leveraging Control Flow Graphs (CFGs) consistently outperform sequential models, although they struggle with data-flow-dependent vulnerabilities like integer overflows [51].

Neuro-symbolic approaches are also making inroads in binary analysis. Frameworks that compress million-node Code Property Graphs into typed Software Supply Chain Knowledge Graphs enable efficient reasoning over stripped binaries, achieving CVE detection rates of 93.3% [41]. Addition-

ally, binary diffing frameworks utilize LLMs to summarize function-level changes, augmented by Functional Sensitivity Scores to automate the triage of malicious code introductions [50].

2.4.2. AI Model Security and Serialization The security of AI models themselves is a critical concern. MalHug, an end-to-end security scanning pipeline, employs semantic-level taint analysis to detect malicious code in model serialization formats like Pickle and Keras, identifying 91 malicious models in a mirror of Hugging Face [24]. The PICKLECLOAK framework demonstrates the stealthiness of such attacks by using Exception-Oriented Programming (EOP) to crash scanners, evading detection in 19 out of 22 model loading paths [52].

To counter these threats, dynamic analysis of model loading and risky function layers is essential. Frameworks that combine static byte-pattern analysis with dynamic payload injection have shown that 96% of unsafe models are exploitable, emphasizing the need for robust deserialization safeguards [22]. Furthermore, research into “Vibe Coding” attacks reveals that poisoned AI training data can target Web3 patterns, necessitating new standards for client-side security [53].

2.5. Governance Frameworks: SBOMs, Provenance, and Standards

Governance frameworks, particularly Software Bills of Materials (SBOMs), are foundational to supply chain security, yet their implementation faces significant hurdles.

2.5.1. SBOM Generation and Quality While SBOM generation tools are abundant, their quality and consistency vary widely. Empirical studies reveal that inter-tool consistency for package detection averages merely 7.84% to 12.77%, and license accuracy remains below 20% [54]. Static parsers often ignore unpinned dependencies and non-standard lockfiles, leading to significant data loss, while dynamic simulators introduce temporal inconsistencies [55]. To address this, kernel-level monitoring frameworks like Bomfather intercept file access events during compilation to generate content-based Merkle trees, capturing dynamically loaded dependencies missed by manifest-based tools [56].

2.5.2. SBOM Consumption and Vulnerability Management The consumption of SBOMs for vulnerability management is plagued by high false positive rates. Downstream scanners exhibit a 97.5% false positive rate when not augmented with reachability analysis [57]. Frameworks like SAVANT integrate LLM-based semantic reachability analysis to prune unreachable vulnerabilities, improving F1-scores by 103% over traditional scanners [58]. Additionally, the integration of VEX (Vulnerability Exploitability eXchange) documents with SBOMs is gaining traction, though maintainers often reject static VEX files due to obsolescence risks, preferring automation within CI pipelines [59].

2.5.3. Provenance and Attestation Provenance and attestation are critical for establishing trust. The TAIBOM framework extends SBOM standards to include cryptographic attestations linking datasets, code, and model weights, enabling verifiable trust chains absent in current Model Cards [60]. Blockchain-enabled SBOM sharing via Verifiable Credentials allows for secure selective disclosure and zero-knowledge verification, although scalability remains a concern [61]. Furthermore, the concept of “Continuous SBOMs” via API-driven metadata updates is being proposed to address the static nature of current implementations [62].

2.5.4. AI Bill of Materials (AIBOM) The unique requirements of AI systems have led to the development of AI Bill of Materials (AIBOM) standards. Extensions to SPDX 3.0 now include modular profiles for AI and Dataset metadata, capturing model provenance, training configurations, and risk metadata [63]. Operationalizing AIBOMs involves embedding AI-specific fields like model weights and hyperparameters into machine-readable schemas, enabling automated vulnerability attribution and bit-identical reproducibility [64]. However, challenges remain in representing dynamic runtime dependencies and ensuring the accuracy of self-reported metadata [65].

3. Challenges and Outlook

Despite significant progress, the field of AI software supply-chain security faces profound challenges that hinder the deployment of robust, scalable defenses.

3.1. Dataset Bias and Generalization

A critical bottleneck is the reliance on biased and synthetic datasets. Many vulnerability detection models are trained on datasets like BigVul or Juliet, which suffer from label noise, class imbalance, and artificial patterns [16], [15]. Studies show that models trained on augmented data often overfit to specific transformation types, failing to generalize to real-world code where vulnerability patterns are subtle and context-dependent [15]. The lack of large-scale, high-quality ground truth datasets for malicious packages and real-world exploits further limits the evaluation of detection systems [66], [67]. Future research must prioritize the construction of diverse, realistic benchmarks that reflect the complexity of modern software ecosystems.

3.2. Adversarial Robustness and Evasion

Detection systems are increasingly vulnerable to adversarial attacks. Malicious actors employ techniques such as code obfuscation, packing, and “concept convergence” to evade ML-based detectors [68], [69]. For instance, problem-space adversarial training can improve detection of obfuscated packages by 10%, but often at the cost of performance on clean samples [70]. Furthermore, attackers are adapting to metadata-based detection by manipulating repository attributes and nurturing accounts over time to bypass reputation systems [71], [72]. Developing detectors that are robust to adaptive adversaries requires a shift towards dynamic, behavior-based analysis and neuro-symbolic verification that is harder to evade [41], [70].

3.3. The SBOM Consumption Gap

While SBOM generation has become commoditized, consumption remains a significant challenge. Industry summits consistently highlight that SBOMs suffer from trust deficits, lack of transitive dependency data, and inconsistent formatting, rendering them difficult to use for actionable risk mitigation [13], [14]. The “SBOM lifestyle” requires a cultural shift towards automated consumption workflows, yet current tools often generate overwhelming noise without context [73], [74]. Bridging this gap requires standardized formats, improved interoperability between tools, and the integration of exploitability context (e.g., EPSS, KEV) to prioritize remediation efforts [75], [57].

3.4. Scalability and Operational Overhead

Many advanced detection frameworks, particularly those involving LLMs or dynamic analysis, incur high operational costs. LLM-based reasoning introduces latency and cost barriers, while dynamic

analysis requires sandboxed environments that may not be feasible for all deployment scenarios [11], [39]. Kernel-level monitoring and eBPF-based tracing, while effective, require elevated privileges and can introduce performance overhead [56], [37]. Future solutions must balance detection accuracy with efficiency, potentially through lightweight models, edge computing, or hybrid approaches that reserve heavy analysis for high-risk candidates [76], [77].

3.5. Emerging Threats in AI and Agentic Systems

As AI agents become more autonomous, they introduce new attack surfaces. “Hallucinating AI Hijacking” attacks exploit context-shifting vulnerabilities to bypass safety guardrails, leading models to recommend malicious libraries [78]. Additionally, the supply chain for AI agents themselves, including skills and tools, is vulnerable to hijacking and injection attacks [79], [80]. Securing these systems requires new paradigms for verifying agent intent, constraining tool usage, and ensuring the integrity of the models driving these agents [81], [82].

3.6. Future Research Directions

Future research should focus on several key areas: 1. **Neuro-Symbolic Verification:** Combining the scalability of neural networks with the soundness of symbolic reasoning to provide provable security guarantees [41], [80]. 2. **Autonomous Remediation:** Developing agentic systems that not only detect vulnerabilities but also autonomously generate and validate patches, reducing the mean time to remediation [11], [83]. 3. **Privacy-Preserving Analysis:** Leveraging zero-knowledge proofs and secure multi-party computation to enable vulnerability scanning without exposing proprietary code or dependency graphs [84], [85]. 4. **Human-AI Collaboration:** Designing interfaces that effectively integrate human expertise with AI-driven insights to mitigate alert fatigue and improve decision-making [86], [87]. 5. **Standardization of AIBOM:** Continuing the development and adoption of AIBOM standards to address the unique provenance and lifecycle requirements of AI systems [63], [64].

4. Conclusion

The landscape of AI software supply-chain security is rapidly evolving, driven by the dual forces of increasing complexity in software dependencies and the transformative potential of AI-driven defense mechanisms. This survey has synthesized a vast body of literature, highlighting the transition from static, rule-based analysis to dynamic, agentic, and neuro-symbolic approaches. While significant advancements have been made in detecting vulnerabilities in source code, binaries, and AI models, critical challenges remain in dataset quality, adversarial robustness, and the operationalization of governance frameworks like SBOMs.

The evidence suggests that no single technique is sufficient; a defense-in-depth strategy combining static analysis, dynamic monitoring, and autonomous agentic reasoning is required to secure the modern supply chain. Furthermore, the integration of human oversight with automated systems remains essential to address the nuances of security decision-making. As the field moves forward, addressing the gaps in generalization, scalability, and standardization will be paramount to building resilient software ecosystems capable of withstanding the sophisticated threats of the future. The journey from reactive vulnerability scanning to proactive, autonomous supply chain assurance is underway, but it demands continued innovation and collaboration across the research and industry communities.

References

- [1] Carmine Cesarano, Martin Monperrus. (2024). The Grand Software Supply Chain of AI Systems. arXiv:2604.27781.
- [2] Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro. (2025). The ML Supply Chain in the Era of Software 2.0: Lessons Learned from Hugging Face. arXiv:2502.04484.
- [3] Piotr Przymus, Thomas Durieux. (2024). Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain Attack. arXiv:2504.17473.
- [4] Raula Gaikovina Kula, Christoph Treude. (2022). In War and Peace: The Impact of World Politics on Software Ecosystems. arXiv:2208.01393.
- [5] Zekun Wu, Seonglae Cho, Umar Mohammed. (2025). LibVulnWatch: A Deep Assessment Agent System and Leaderboard for Uncovering Hidden Vulnerabilities in Open-Source AI Libraries. arXiv:2505.08842.
- [6] Zi Liang, Qipeng Xie, Jun He. (2025). Argus: Reorchestrating Static Analysis via a Multi-Agent Ensemble for Full-Chain Security Vulnerability Detection. arXiv:2604.06633.
- [7] César Soto-Valero, Martin Monperrus, Benoit Baudry. (2022). The Multibillion Dollar Software Supply Chain of Ethereum. arXiv:2202.07029.
- [8] Ahmed Zerouali, Tom Mens, Alexandre Decan. (2020). On the Impact of Security Vulnerabilities in the npm and RubyGems Dependency Networks. arXiv:2106.06747.
- [9] Samiha Shimmi, Hamed Okhravi, Mona Rahimi. (2024). AI-Based Software Vulnerability Detection: A Systematic Literature Review. arXiv:2506.10280.
- [10] Nima Shiri Harzevili, Alvine Boaye Belle, Junjie Wang. (2018). A Survey on Automated Software Vulnerability Detection Using Machine Learning and Deep Learning. arXiv:2306.11673.
- [11] Toqeer Ali Syed, Mohammad Riyaz Belgaum, Salman Jan. (2025). Agentic AI for Autonomous Defense in Software Supply Chain Security: Beyond Provenance to Vulnerability Mitigation. arXiv:2512.23480.
- [12] Takaaki Toda, Tatsuya Mori. (2025). CHASE: LLM Agents for Dissecting Malicious PyPI Packages. arXiv:2601.06838.
- [13] Imranur Rahman, Yasemin Acar, Michel Cukier. (2024). S3C2 Summit 2024-09: Industry Secure Supply Chain Summit. arXiv:2505.10538.
- [14] Greg Tystahl, Yasemin Acar, Michel Cukier. (2024). S3C2 Summit 2024-03: Industry Secure Supply Chain Summit. arXiv:2405.08762.
- [15] Niklas Risse, Marcel Böhme. (2024). Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection. arXiv:2306.17193.
- [16] Niklas Risse, Jing Liu, Marcel Bohme. (2025). Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection. arXiv:2408.12986.
- [17] Yanzhe Hu, Shenao Wang, Tianyuan Nie. (2025). Understanding Large Language Model Supply Chain: Structure, Domain, and Vulnerabilities. arXiv:2504.20763.

- [18] Jacob Mahon, Chenxi Hou, Zhihao Yao. (2024). PyPitfall: Dependency Chaos and Software Supply Chain Vulnerabilities in Python. arXiv:2507.18075.
- [19] Laura Baird, Armin Moin. (2025). Cascaded Vulnerability Attacks in Software Supply Chains. arXiv:2601.20158.
- [20] Laura Baird, Armin Moin. (2026). Towards Predicting Multi-Vulnerability Attack Chains in Software Supply Chains from Software Bill of Materials Graphs. arXiv:2604.04977.
- [21] Luiz Giovanini, Daniela Oliveira, Huascar Sanchez. (2024). Leveraging Team Dynamics to Predict Open-source Software Projects’ Susceptibility to Social Engineering Attacks. arXiv:2106.16067.
- [22] Beatrice Casey, Joanna C. S. Santos, Mehdi Mirakhorli. (2024). A Large-Scale Exploit Instrumentation Study of AI/ML Supply Chain Attacks in Hugging Face Models. arXiv:2410.04490.
- [23] Mohamed Nabeel, Oleksii Starov. (2024). Deep Dive into the Abuse of DL APIs to Create Malicious AI Models and How to Detect Them. arXiv:2601.04553.
- [24] Jian Zhao, Shenao Wang, Yanjie Zhao. (2024). Models Are Codes: Towards Measuring Malicious Code Poisoning Attacks on Pre-trained Model Hubs. arXiv:2409.09368.
- [25] Shuhan Liu, Xing Hu, Xin Xia. (2024). An Empirical Study of Vulnerable Package Dependencies in LLM Repositories. arXiv:2508.21417.
- [26] Huaifeng Zhang, Fahmi Abdulqadir Ahmed, Dyako Fatih. (2024). MACHINE LEARNING SYSTEMS ARE BLOATED AND VULNERABLE. arXiv:2212.09437.
- [27] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid. (2022). What are Weak Links in the npm Supply Chain?. arXiv:2112.10165.
- [28] Fabio Massacci, Ivan Pashchenko. (2021). Technical Leverage in a Software Ecosystem: Development Opportunities and Security Risks. arXiv:2103.03317.
- [29] Kelechi G. Kalu, Sofia Okorafor, Betul Durak. (2024). ARMS: A Vision for Actor Reputation Metric Systems in the Open-Source Software Supply Chain. arXiv:2505.18760.
- [30] Wenxin Jiang, Berk Çakar, Mikola Lysenko. (2025). ConfuGuard: Using Metadata to Detect Active and Stealthy Package Confusion Attacks Accurately and at Scale. arXiv:2502.20528.
- [31] Amanpreet Singh Saimbhi. (2024). Enhancing Software Vulnerability Detection Using Code Property Graphs and Convolutional Neural Networks. arXiv:2503.18175.
- [32] Sahil Suneja, Yunhui Zheng, Yufan Zhuang. (2019). Learning to map source code to software vulnerability using code-as-a-graph. arXiv:2006.08614.
- [33] Rijha Safdar, Danyail Mateen, Syed Taha Ali. (2025). Data and Context Matter: Towards Generalizing AI-based Software Vulnerability Detection. arXiv:2508.16625.
- [34] Chaomeng Lu, Bert Lagasse. (2025). From Lab to Reality: A Practical Evaluation of Deep Learning Models and LLMs for Vulnerability Detection. arXiv:2512.10485.
- [35] Alejandro Mazuera-Rozo, Anamaria Mojica-Hanke, Mario Linares-Vasquez. (2021). Shallow or Deep? An Empirical Study on Detecting Vulnerabilities using Deep Learning. arXiv:2103.11940.
- [36] Yuejun Guo, Qiang Hu, Qiang Tang. (2024). An Empirical Study of the Imbalance Issue in Software Vulnerability Detection. arXiv:2602.12038.

- [37] Sk Tanzir Mehedi, Raja Jurdak, Chadni Islam. (2025). eDySec: A Deep Learning-based Explainable Dynamic Analysis Framework for Detecting Malicious Packages in PyPI Ecosystem. arXiv:2604.26219.
- [38] Sk Tanzir Mehedi, Chadni Islam, Gowri Ramachandran. (2024). DySec: A Machine Learning-based Dynamic Analysis for Detecting Malicious Packages in PyPI Ecosystem. arXiv:2503.00324.
- [39] Sk Tanzir Mehedi, Raja Jurdak, Chadni Islam. (2024). QUT-DV25: A Dataset for Dynamic Analysis of Next-Gen Software Supply Chain Attacks. arXiv:2505.13804.
- [40] Xinyi Zheng, Chen Wei, Shenao Wang. (2024). Towards Robust Detection of Open Source Software Supply Chain Poisoning Attacks in Industry Environments. arXiv:2409.09356.
- [41] Bowei Ning, Xuejun Zong, Lian Lian. (2025). SECURING THE DARK MATTER: A SEMANTIC-ENHANCED NEURO-SYMBOLIC FRAMEWORK FOR SUPPLY CHAIN ANALYSIS OF OPAQUE INDUSTRIAL SOFTWARE. arXiv:2605.07737.
- [42] Dang-Khoa Nguyen, Gia-Thang Ho, Quang-Minh Pham. (2024). Taint-Based Code Slicing for LLMs-based Malicious NPM Package Detection. arXiv:2512.12313.
- [43] Xiao Chen. (2024). CrossInspector: A Static Analysis Approach for Cross-Contract Vulnerability Detection. arXiv:2408.15292.
- [44] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko. (2024). Leveraging Large Language Models to Detect npm Malicious Packages. arXiv:2403.12196.
- [45] XiangRui Zhang, XueJie Du, HaoYu Chen. (2024). Automatically Generating Rules of Malicious Software Packages via Large Language Model. arXiv:2504.17198.
- [46] Md Mahbubur Rahman, Monoshi Kumar Roy, Mirza Sanjida Alam. (2024). TO ERR IS MACHINE: VULNERABILITY DETECTION CHALLENGES LLM REASONING. arXiv:2403.17218.
- [47] Motunrayo Ibiyo, Thinakone Louangdy, Phuong T. Nguyen. (2025). Detecting Malicious Source Code in PyPI Packages with LLMs: Does RAG Come in Handy?. arXiv:2504.13769.
- [48] Renwei Meng, Haoyi Wu, Jingming Wang. (2024). VulnAgent-X: A Layered Agentic Framework for Repository-Level Vulnerability Detection. arXiv:2603.13384.
- [49] Tanmay Singla, Berk Çakar, Paschal C. Amusuo. (2024). Towards a Benchmark for Dependency Decision-Making. arXiv:2601.00205.
- [50] Meet Udeshi, Venkata Sai Charan Putrevu, Prashanth Krishnamurthy. (2025). BINARY DIFF SUMMARIZATION USING LARGE LANGUAGE MODELS. arXiv:2509.23970.
- [51] Mitchell Petingola. (2024). Deep Learning-based Binary Analysis for Vulnerability Detection in x86-64 Machine Code. arXiv:2601.09157.
- [52] Tong Liu, Guozhu Meng, Peng Zhou. (2024). The Art of Hide and Seek: Making Pickle-Based Model Supply Chain Poisoning Stealthy Again. arXiv:2508.19774.
- [53] Martin Monperrus, Raphina Liu, Monica Jin. (2025). Software Supply Chain Security of Web3. arXiv:2511.12274.
- [54] Chengjie Wang, Jingzheng Wu, Hao Lyu. (2026). A Large Scale Empirical Analysis on the Adherence Gap between Standards and Tools in SBOM. arXiv:2601.05622.

- [55] Giacomo Benedetti, Serena Cofano, Matteo Dell’Amico. (2024). SBOM Generation Tools in the Python Ecosystem: an In-Detail Analysis. arXiv:2409.01214.
- [56] Naveen Srinivasan, Nathan Naveen, Neil Naveen. (2025). Bomfather: An eBPF-based Kernel-level Monitoring Framework for Accurate Identification of Unknown, Unused, and Dynamically Loaded Dependencies in Modern Software Supply Chains. arXiv:2503.02097.
- [57] Li Zhou, Marc Dacier, Charalambos Konstantinou. (2025). A Reality Check on SBOM-based Vulnerability Management: An Empirical Study and A Path Forward. arXiv:2511.20313.
- [58] Wang Lingxiang, Quanzhi Fu, Wenjia Song. (2024). SAVANT: Vulnerability Detection in Application Dependencies through Semantic-Guided Reachability Analysis. arXiv:2506.17798.
- [59] Davide Fucci, Massimiliano Di Penta, Simone Romano. (2025). Augmenting Software Bills of Materials with Software Vulnerability Description: A Preliminary Study on GitHub. arXiv:2503.13998.
- [60] Vadim Safronov, Anthony McCaigue, Nicholas Allott. (2024). TAIBOM: Bringing Trustworthiness to AI-Enabled Systems. arXiv:2510.02169.
- [61] Boming Xia, Dawen Zhang, Yue Liu. (2024). TRUST IN SOFTWARE SUPPLY CHAINS: BLOCKCHAIN-ENABLED SBOM AND THE AIBOM FUTURE. arXiv:2307.02088.
- [62] Sivana Hamer, Pat Morrison, William Enck. (2025). S3C2 Summit 2025-07: Government Secure Supply Chain Summit. arXiv:2605.29140.
- [63] Gopi Krishnan Rajbahadur, Keheliya Gallaba, Elyas Rashno. (2026). Building an Open AIBOM Standard in the Wild: An Experience Report on Extending the SPDX SBOM (ISO/IEC 5962:2021) for AI Supply Chains. arXiv:2510.07070.
- [64] Petar Radanliev, Omar Santos, Carsten Maple. (2026). Operationalising Artificial Intelligence Bills of Materials (AIBOMs) for Verifiable AI Provenance and Lifecycle Assurance. arXiv:2605.19755.
- [65] Shuai Zhang, Minzhao Lyu, Hassan Habibi Gharakheili. (2026). A Survey on Mapping Digital Systems with Bill of Materials: Development, Practices, and Challenges. arXiv:2601.11678.
- [66] Nasif Imtiaz, Seaver Thorn, Laurie Williams. (2021). A comparative study of vulnerability reporting by software composition analysis tools. arXiv:2108.12078.
- [67] Xiaoyan Zhou, Ying Zhang, Wenjia Niu. (2024). An Analysis of Malicious Packages in Open-Source Software in the Wild. arXiv:2404.04991.
- [68] Daniel Gibert, Nikolaos Totosis, Constantinos Patsakis. (2024). ASSESSING THE IMPACT OF PACKING ON MACHINE LEARNING-BASED MALWARE DETECTION AND CLASSIFICATION SYSTEMS. arXiv:2410.24017.
- [69] Wenbo Guo, Zhongwen Chen, Zhengzi Xu. (2025). Understanding NPM Malicious Package Detection: A Benchmark-Driven Empirical Analysis. arXiv:2603.27549.
- [70] Biagio Montaruli, Luca Compagna, Serena Elisa Ponta. (2025). One Detector Fits All: Robust and Adaptive Detection of Malicious Packages from PyPI to Enterprises. arXiv:2512.04338.
- [71] Yu Li, Wei Ma, Zhi Chen. (2024). AgentGuard: A Multi-Agent Framework for Robust Package Confusion Detection via Hybrid Search and Metadata-Content Fusion. arXiv:2604.16309.

- [72] Biagio Montaruli, Serena Elisa Ponta, Luca Compagna. (2025). SourceBroken: A large-scale analysis on the (un)reliability of SourceRank in the PyPI ecosystem. arXiv:2512.24400.
- [73] Mindy Tran, Yasemin Acar, Michel Cukier. (2022). S3C2 Summit 2022-09: Industry Secure Supply Chain Summit. arXiv:2307.15642.
- [74] Eric O’Donoghue, Yvette Hastings, Ernesto Ortiz. (2024). Software Bill of Materials in Software Supply Chain Security: A Systematic Literature Review. arXiv:2506.03507.
- [75] Abdurrahman Tolay. (2026). Automated SBOM-Driven Vulnerability Triage for IoT Firmware: A Lightweight Pipeline for Risk Prioritization. arXiv:2601.01308.
- [76] Chun Yin Chiu. (2024). Lightweight Vulnerability Detection from Code Metrics and Token Features. arXiv:2605.04260.
- [77] Miles Farmer, Ekinan Ufuktepe, Anne Watson. (2024). Software Vulnerability Detection Using a Lightweight Graph Neural Network. arXiv:2603.29216.
- [78] David Noever, Forrest McKee. (2024). Hallucinating AI Hijacking Attack: Large Language Models and Malicious Code Recommenders. arXiv:2410.06462.
- [79] Florian Holzbauer, David Schmidt, Gabriel Gegenhuber. (2026). Malicious Or Not: Adding Repository Context to Agent Skill Classification. arXiv:2603.16572.
- [80] Shenao Wang, Junjie He, Yanjie Zhao. (2026). Elementary, My Dear Watson.” Detecting Malicious Skills via Neuro-Symbolic Reasoning across Heterogeneous Artifacts. arXiv:2603.27204.
- [81] Lisa Thiergart, Yoav Tzfati, Peter Wagstaff. (2025). SL5 Standard for AI Security. arXiv:2605.08449.
- [82] Szilard Enyedi. (2025). Human-Certified Module Repositories for the AI Age. arXiv:2603.02512.
- [83] Maria Camporese, Fabio Massacci. (2025). Using ML filters to help automated vulnerability repairs: when it helps and when it doesn’t. arXiv:2504.07027.
- [84] Tom Sorger, Eric Cornelissen, Aman Sharma. (2026). zkSBOM: Privacy-Preserving SBOM Sharing with Zero-Knowledge Sets. arXiv:2605.00076.
- [85] Eman Abu Ishgair, Chinenye Okafor, Marcela S. Melara. (2024). Trustworthy and Confidential SBOM Exchange. arXiv:2509.13217.
- [86] Raula Gaikovina Kula. (2024). Reducing Alert Fatigue via AI-Assisted Negotiation: A Case for Dependabot. arXiv:2502.06175.
- [87] Jessy Ayala, Yu-Jye Tung, Joshua Garcia. (2024). A Mixed-Methods Study of Open-Source Software Maintainers On Vulnerability Management and Platform Security Features. arXiv:2409.07669.